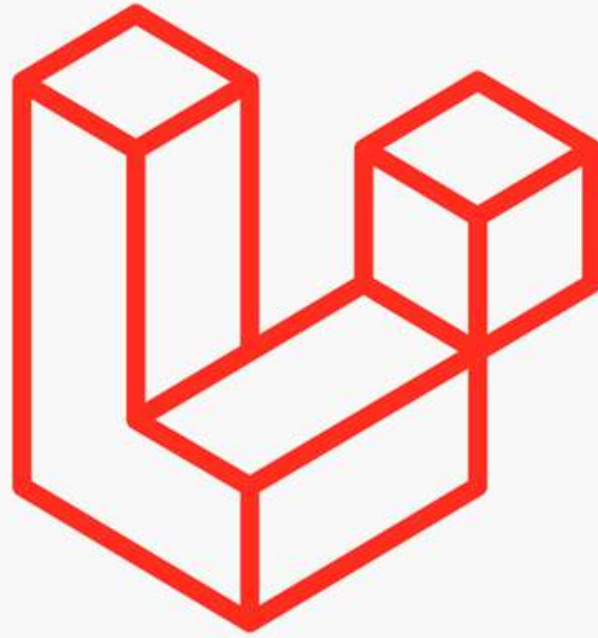




Laravel

Framework?

A web framework is a software platform that provides a structured and standardized way to build web applications.

Popular Web Frameworks

Top 10 Backend Frameworks

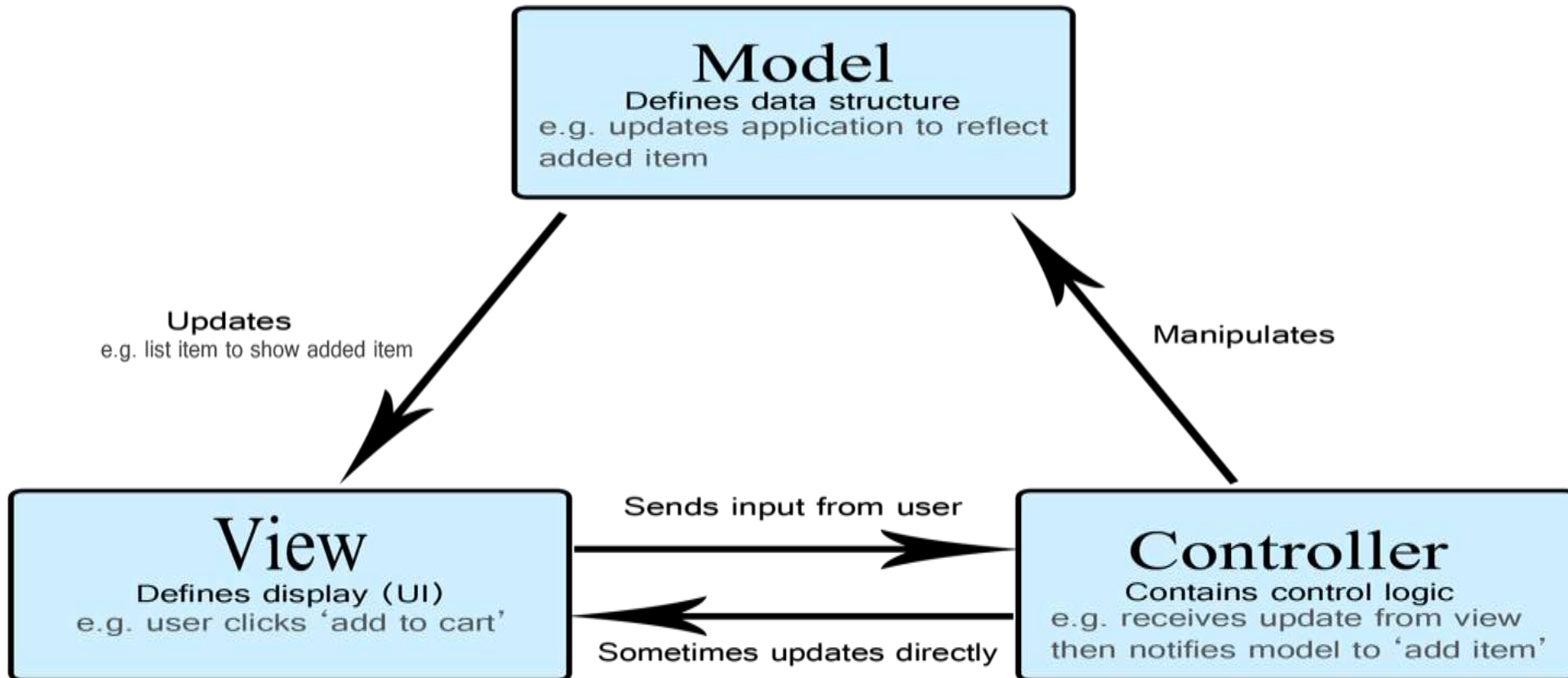
django

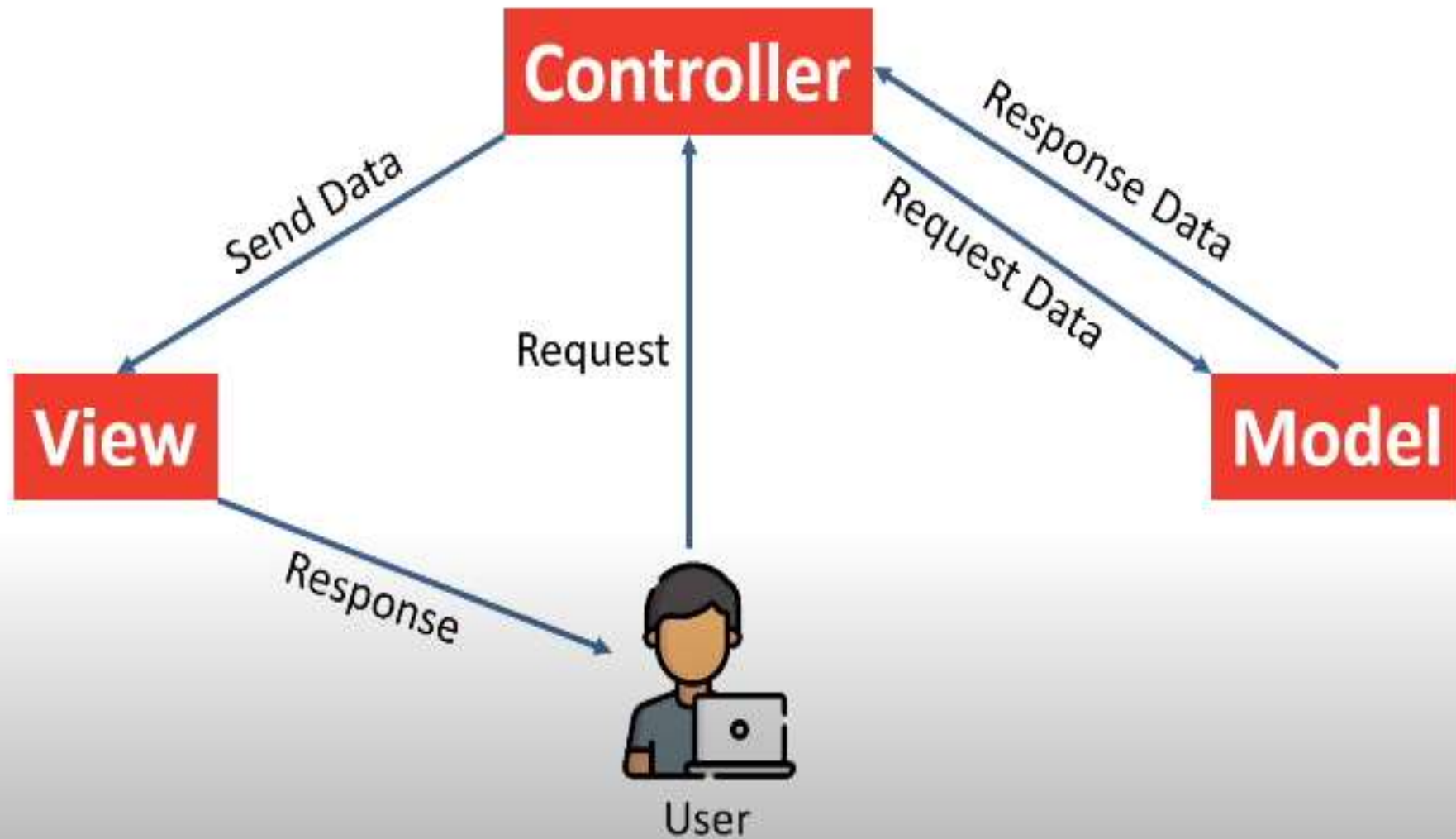


koa



What is MVC ?





Benefits of MVC

- Organized Code
- Independent Block
- Reduces the complexity of Web Applications
- Easy to maintain
- Easy to modify
- Code reusability
- Improved collaboration
- Platform independence

Laravel

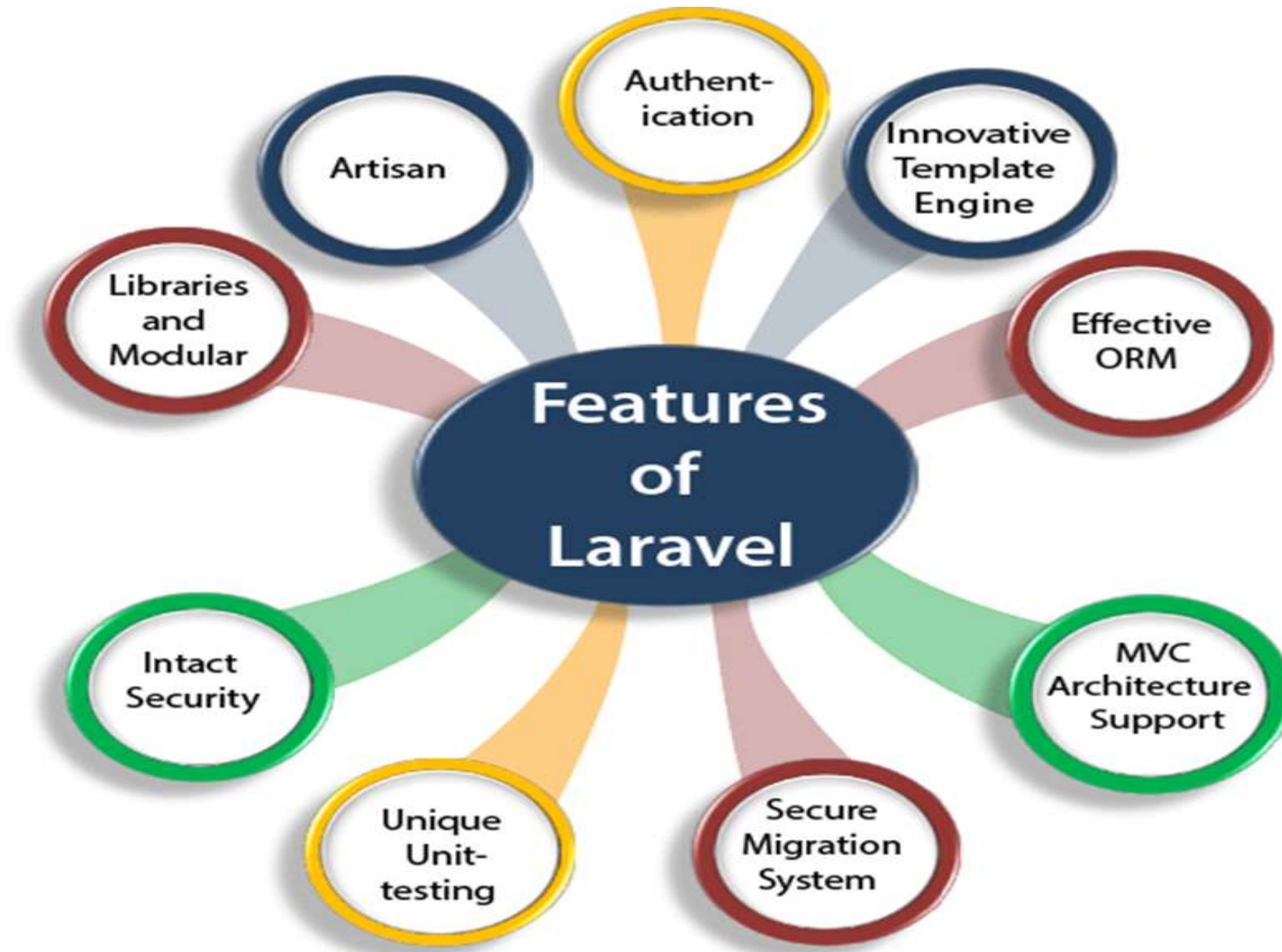
Laravel is a PHP-based web framework for building high-end web applications using its significant and graceful syntaxes.

Taylor Otwell developed Laravel in July 2011, and it was released more than five years after the release of the Codeigniter framework.



“Laravel is the strongest contender in the PHP ecosystem simply because it includes the features needed to build modern web applications.

Taylor Otwell,
Creator of Laravel, Forge, Envoyer



Authentication

Authentication is the most important factor in a web application, and developers need to spend a lot of time writing the authentication code. Laravel contains an **inbuilt authentication system**, you only need to configure models, views, and controllers to make the application work.

Innovative Template Engine

Laravel's innovative template engine, called **Blade**, is designed to simplify the process of creating and managing views (templates) in web applications. Blade provides a clean, expressive syntax that allows developers to write views with ease while providing powerful features for dynamic content rendering.

Effective Object Relational Mapping

When building applications, developers often use object-oriented programming languages like Python, Java, or php to represent entities and their relationships. **These entities are typically classes that have attributes and methods. However, databases primarily store data in tables with rows and columns.**

ORMs provide a solution to this problem by abstracting the database interactions and mapping the objects to database tables, thereby enabling developers to work with their preferred object-oriented paradigm without worrying about the underlying database structure.

```
@Entity
@Table(name="addresses")
public class Address {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="id")
    private Long id;

    @Column(name="street")
    private String street;

    @Column(name="city")
    private String city;

    @Column(name="state")
    private String state;

    @Column(name="country")
    private String country;

    @Column(name="zip_code")
    private String zipCode;
```

addresses	
id	BIGINT
city	VARCHAR(255)
country	VARCHAR(255)
state	VARCHAR(255)
street	VARCHAR(255)
zip_code	VARCHAR(255)
order_id	BIGINT
Indexes	

```
Address address = new Address();
address.setCity("Pune");
address.setStreet("Kothrud");
address.setState("Maharashtra");
address.setCountry("India");
address.setZipCode("411047");
```

Result Grid						
Filter Rows: Search						
id	city	country	state	street	zip_code	
2	Pune	India	Maharashtra	Kothrud	411047	
3	Pune	India	Maharashtra	Kothrud	411047	

Laravel's support for the Model-View-Controller (MVC) architectural pattern brings several benefits to web application development. MVC is a software design pattern that **separates the application logic into three interconnected components: Model, View, and Controller.**

Secure Migration System

Laravel's migration system **simplifies the process of managing database changes in a structured and consistent manner.** It promotes version control of the database schema and helps developers work collaboratively, ensuring the security and integrity of the application's database.

Unique Unit Testing

Laravel seamlessly integrates with **PHPUnit**, one of the most widely used and well-established testing frameworks in the PHP ecosystem. PHPUnit offers a rich set of assertion methods and testing features, making it easier to write clear and concise tests.

Laravel Dusk, the built-in browser testing tool, provides an excellent way to perform end-to-end tests and **interact with your application as a real user would**.

Intact Security

Laravel uses **bcrypt hashing** to securely store user passwords. Bcrypt is a strong and slow hashing algorithm, making it difficult for attackers to crack passwords using brute-force attacks.

Libraries and Modules

Laravel is very popular, as **some object-oriented libraries and pre-installed libraries are added in this framework; these pre-installed libraries are not added in other PHP frameworks.** One of the most popular libraries is an **authentication library that contains some useful features such as password reset, monitoring active users, bcrypt hashing, and CSRF protection.** This framework is divided into several modules that follow the PHP principles, allowing the developers to build responsive and modular apps.

Artisan

The Laravel framework provides a built-in tool for the command line known as Artisan that **performs the repetitive programming tasks** that do not allow the PHP developers to perform manually. These artisans can also be **used to create the skeleton code, database structure, and its migration**, so it makes it easy to manage the database of the system. It also **generates the MVC files through the command line. Artisan also allows the developers to create their own commands.**

Requirements for Laravel project

- Xampp (supporting php > 8.2)
- Composer
- VS-Code

GO to apachefriends.org



The screenshot shows the homepage of the Apache Friends website. The browser's address bar displays "apachefriends.org". The navigation bar includes links for "Apache Friends", "Download", "Add-ons", "Hosting", "Community", and "About", along with a search bar and a language selector set to "EN".

XAMPP Apache + MariaDB + PHP + Perl

What is XAMPP?
XAMPP is the most popular PHP development environment.

XAMPP is a completely free, easy to install Apache distribution containing MariaDB, PHP, and Perl. The XAMPP open source package has been set up to be incredibly easy to install and to use.


XAMPP

Download
Click here for other versions

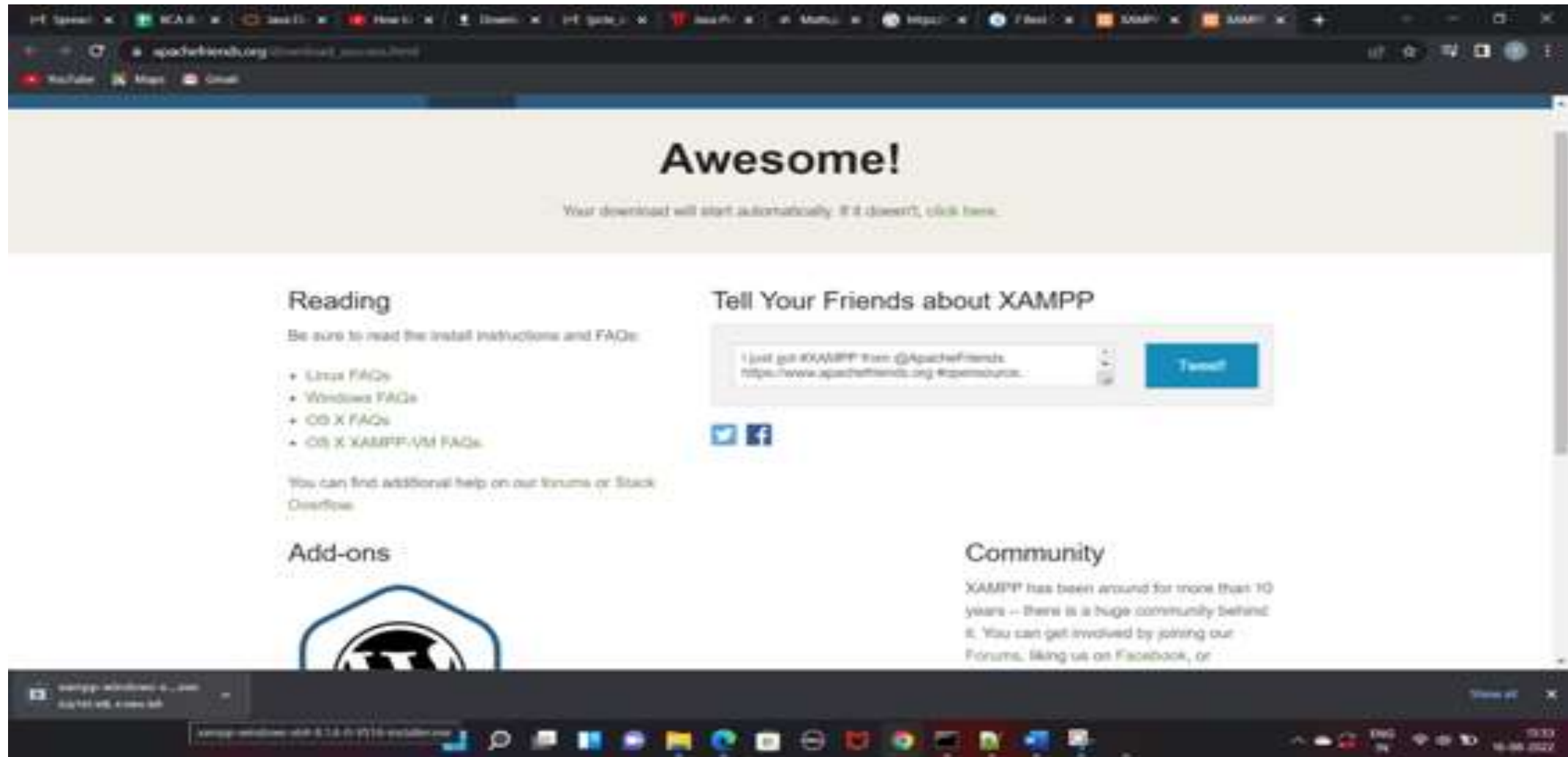
 XAMPP for Windows 0.1.0 (PHP 5.1.6)	 XAMPP for Linux 0.1.0 (PHP 5.1.6)	 XAMPP for OS X 0.1.0 (PHP 5.1.6)
---	--	---

Apache Friends Hi Apache Friends!

Go To XAMPP for Windows



XAMPP will start downloading

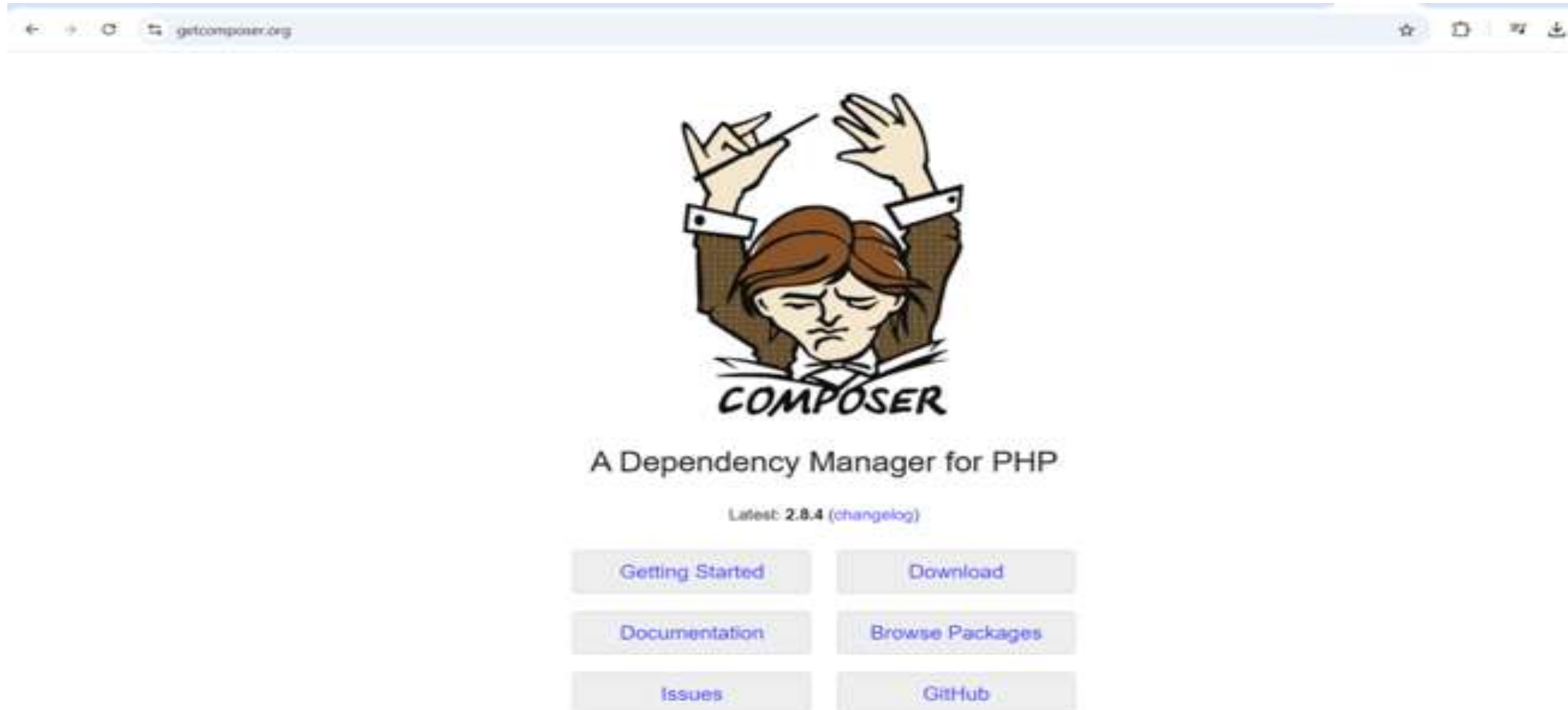


Composer

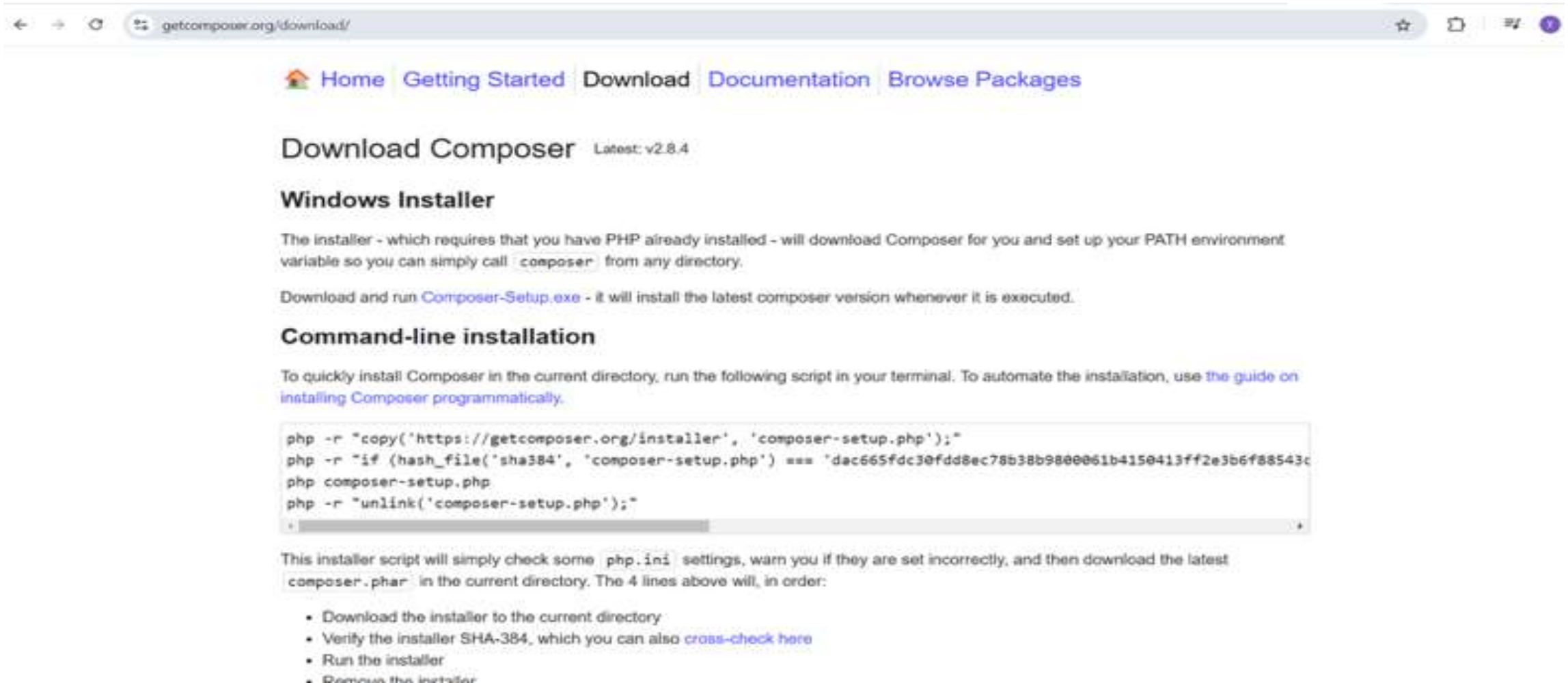


- For managing the **project's dependencies**.
- Its main purpose is to simplify the process of **including external libraries and packages** into your Laravel application.

Go to getcomposer.org



After clicking on Downloads run Composer-setup.exe file



The screenshot shows the 'Download Composer' page on the official website. It includes navigation links, a 'Download Composer' section with the latest version (v2.8.4), a 'Windows Installer' section explaining the installer's requirements and usage, and a 'Command-line installation' section with a terminal script and instructions.

getcomposer.org/download/

Home Getting Started Download Documentation Browse Packages

Download Composer Latest: v2.8.4

Windows Installer

The installer - which requires that you have PHP already installed - will download Composer for you and set up your PATH environment variable so you can simply call `composer` from any directory.

Download and run [Composer-Setup.exe](#) - it will install the latest composer version whenever it is executed.

Command-line installation

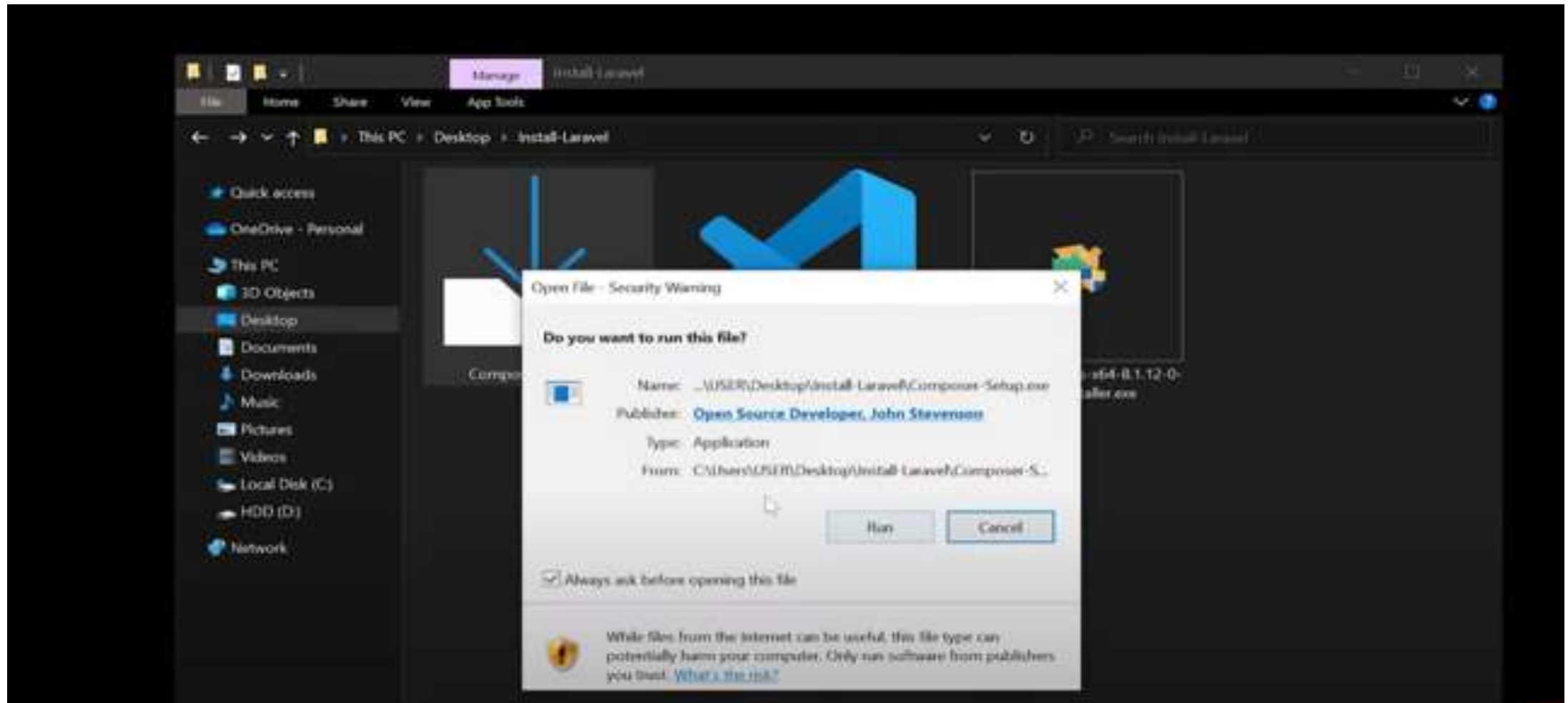
To quickly install Composer in the current directory, run the following script in your terminal. To automate the installation, use [the guide on installing Composer programmatically](#).

```
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
php -r "if (hash_file('sha384', 'composer-setup.php') === 'dac665fdc3@fdd8ec78538b98@0061b4150413ff2e3b6f88543c
php composer-setup.php
php -r "unlink('composer-setup.php');"
```

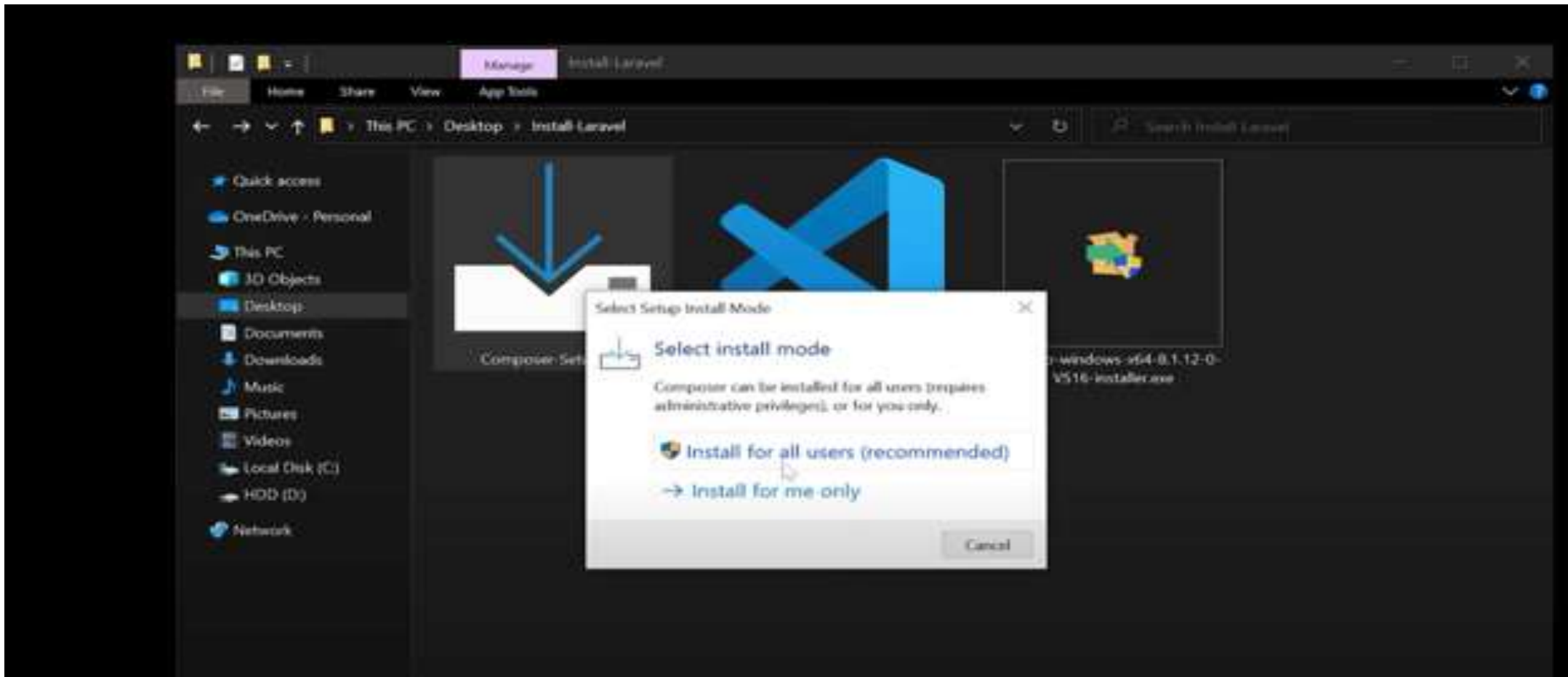
This installer script will simply check some `php.ini` settings, warn you if they are set incorrectly, and then download the latest `composer.phar` in the current directory. The 4 lines above will, in order:

- Download the installer to the current directory
- Verify the installer SHA-384, which you can also [cross-check here](#)
- Run the installer
- Remove the installer

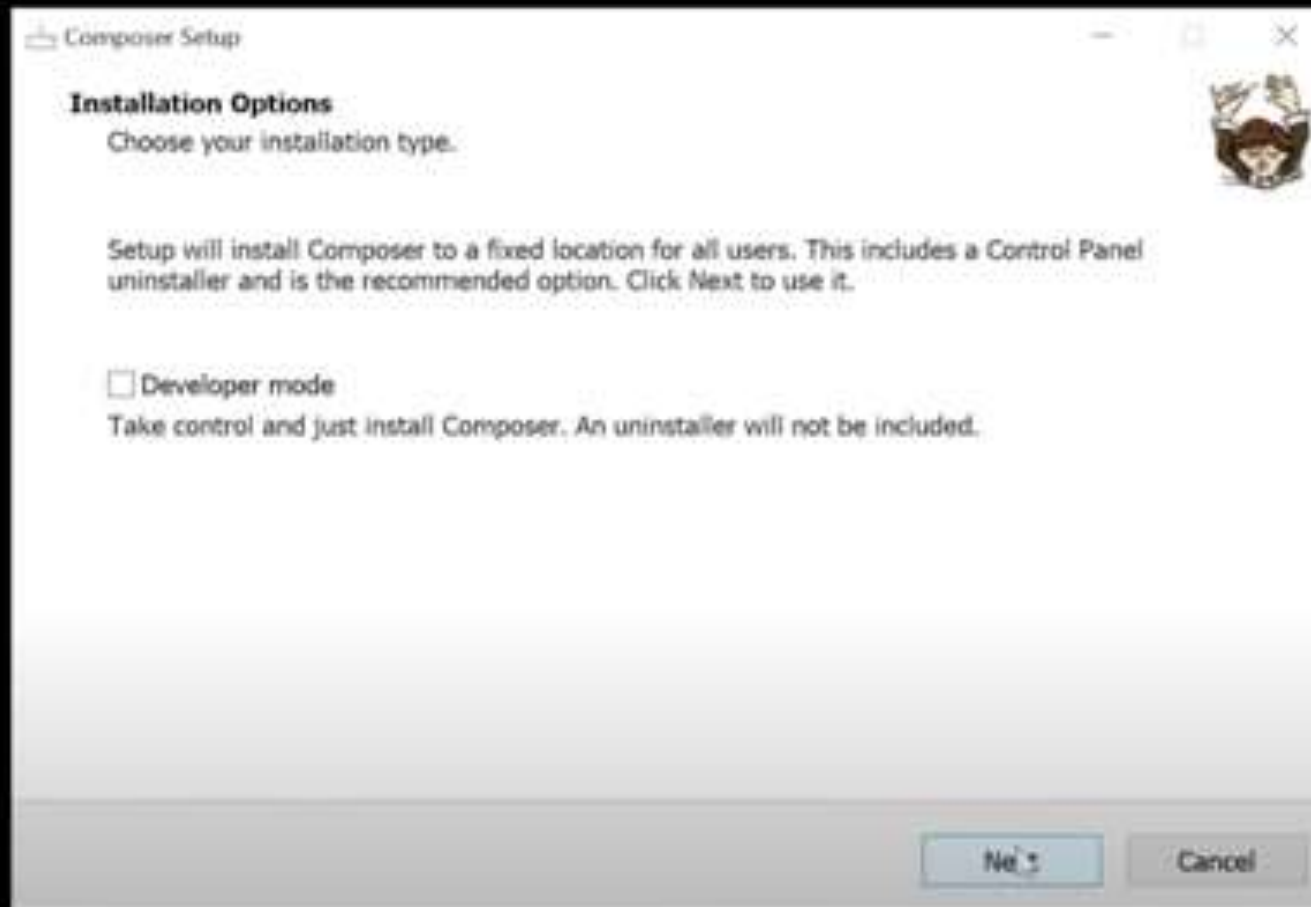
Click on Run



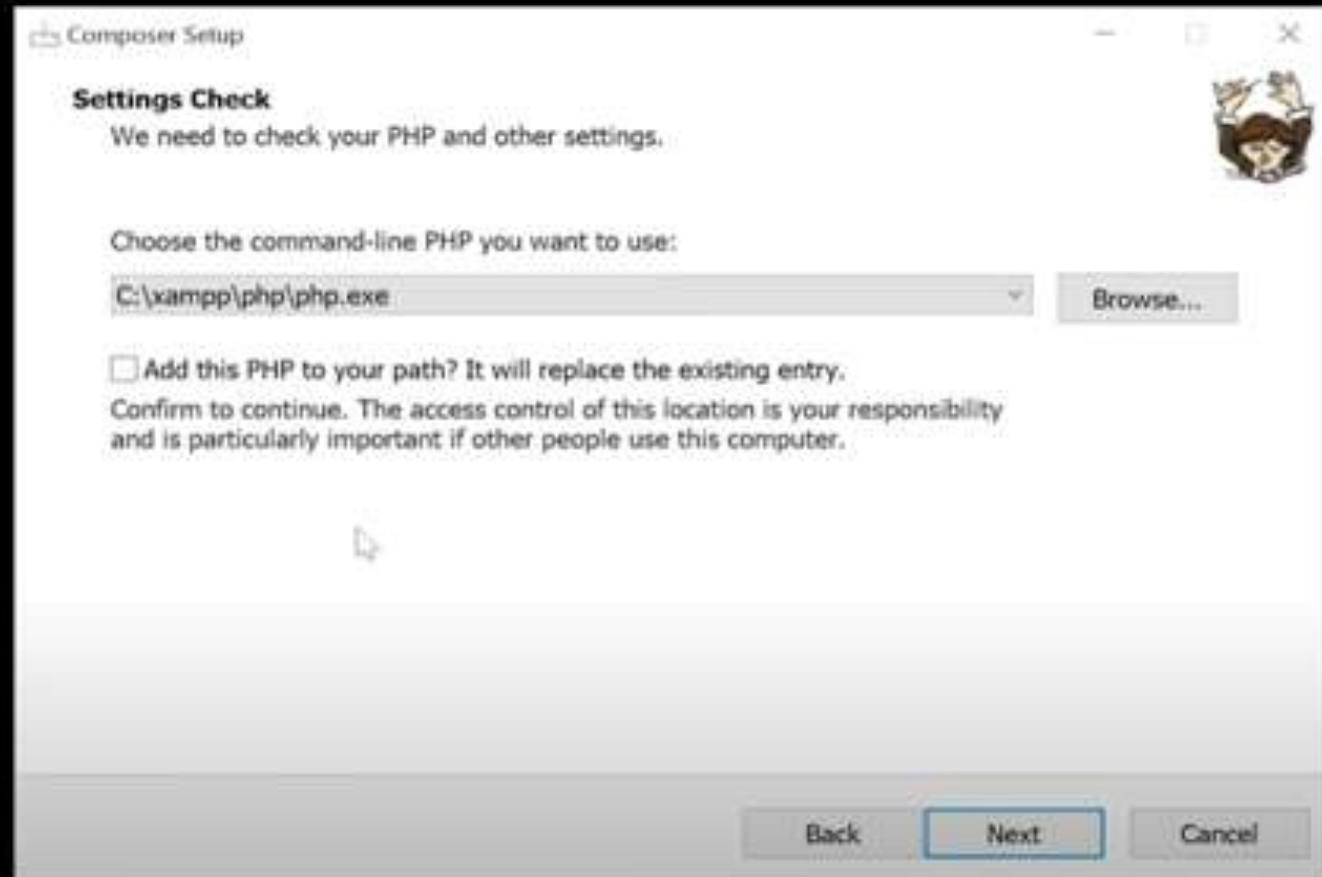
Click on Install for all users



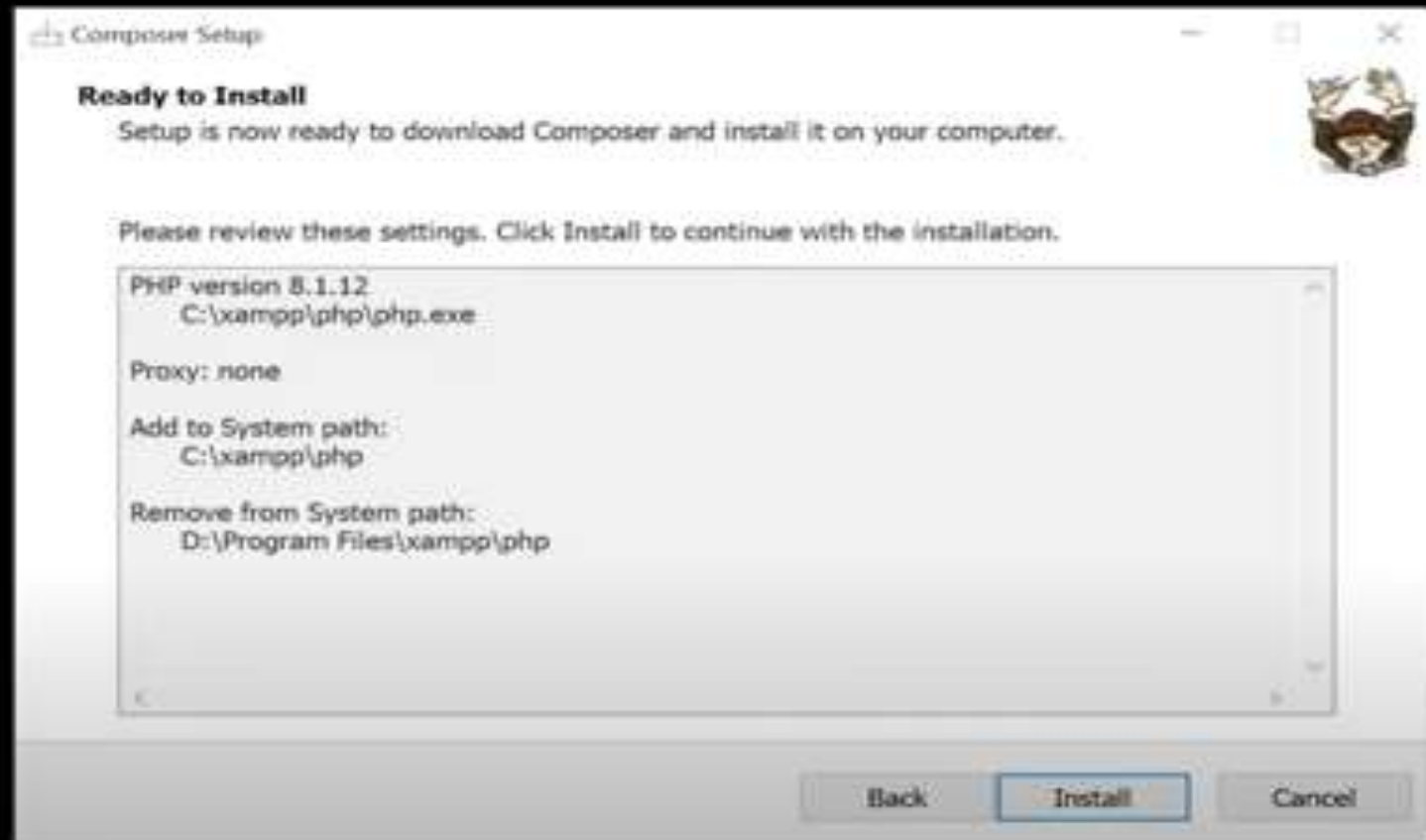
Click on Next



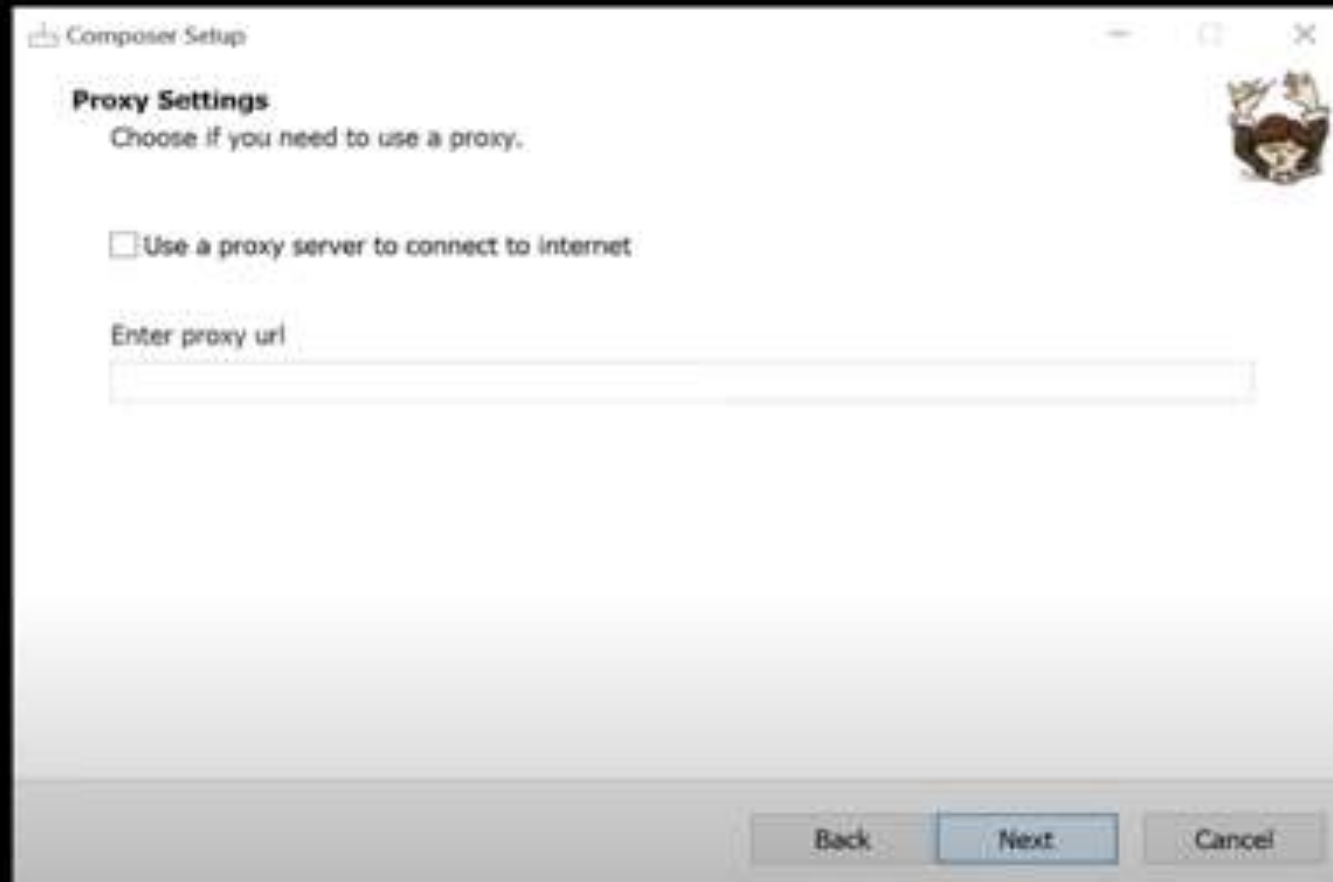
Click on Check Box and click on Next



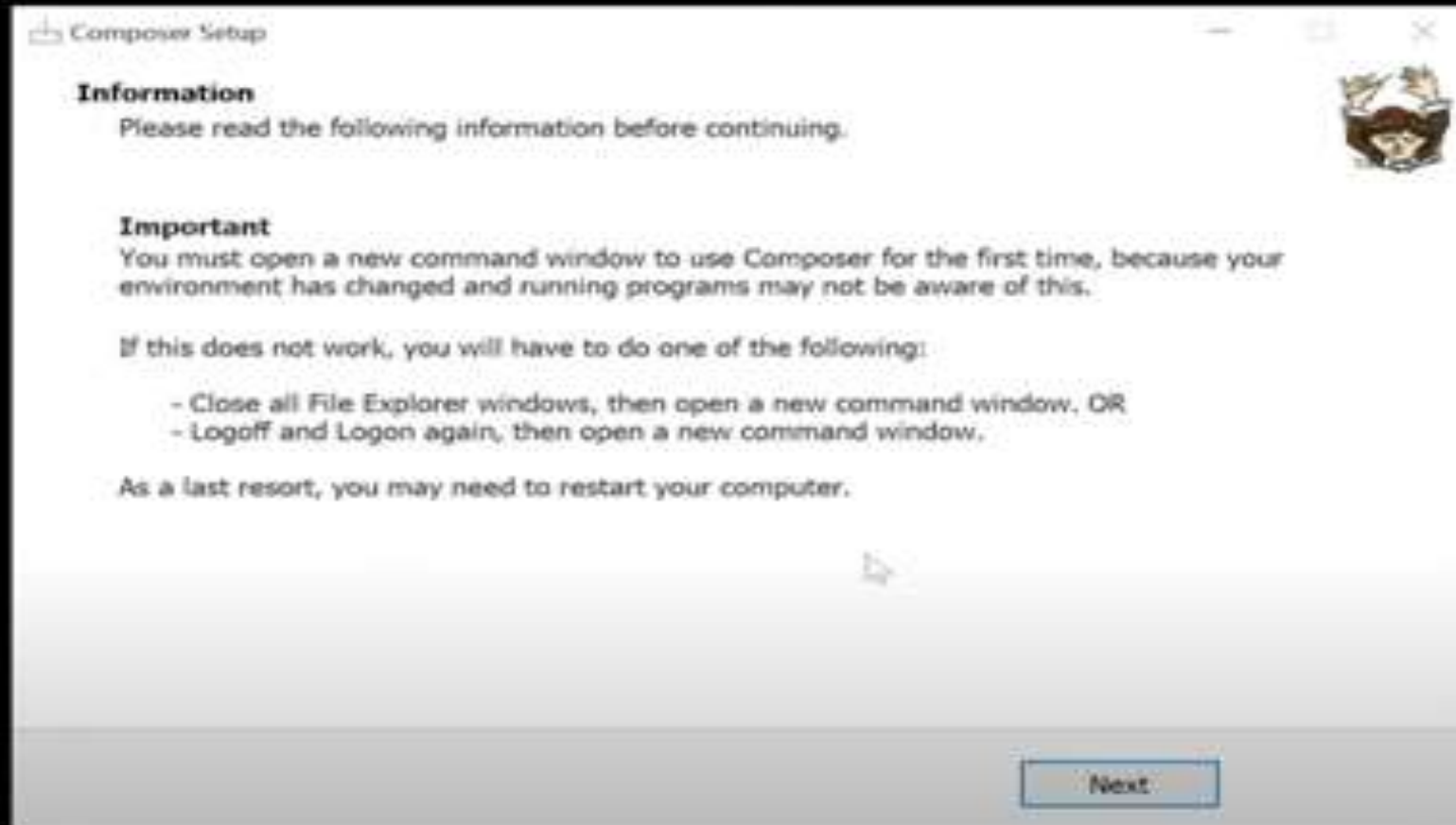
Click on Install



Click on Next



Click Next



Check PHP Version

```
php -v
```

Check Git Version

```
git --version
```

Check Composer Version

```
composer --version
```

Or

```
composer
```

Check Laravel version

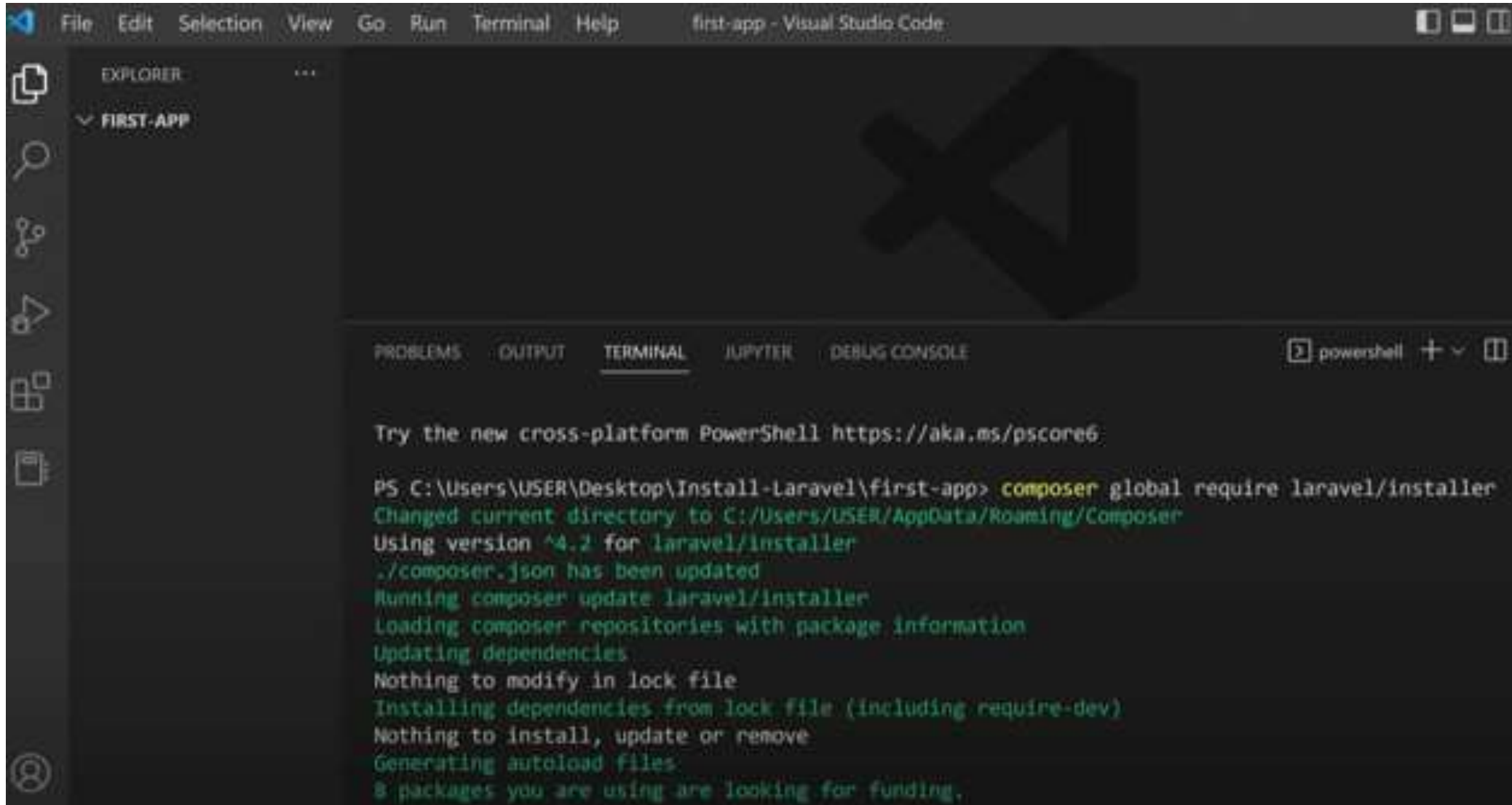
```
laravel -v
```

Installation of Laravel

- Global Installation
 - 1- **composer global require laravel/installer** (one-time command)
 - 2- **laravel new first-app** (for new app)
- Per-project Installation

composer create-project laravel/laravel first-app (every time for new app)

Open the folder in VS Code and run the command
(Terminal New Terminal)



The screenshot shows the Visual Studio Code interface with a terminal window open. The terminal is running a PowerShell session. The command executed is `composer global require laravel/installer`. The output shows that the current directory was changed to the Composer global directory, version 4.2 was used, and the `composer.json` file was updated. The terminal also shows the process of updating dependencies and installing them from the lock file.

```
Try the new cross-platform PowerShell https://aka.ms/powershell

PS C:\Users\USER\Desktop\Install-Laravel\first-app> composer global require laravel/installer
Changed current directory to C:/Users/USER/AppData/Roaming/Composer
Using version ^4.2 for laravel/installer
./composer.json has been updated
Running composer update laravel/installer
Loading composer repositories with package information
Updating dependencies
Nothing to modify in lock file
Installing dependencies from lock file (including require-dev)
Nothing to install, update or remove
Generating autoload files
8 packages you are using are looking for funding.
```

Issues and solutions- When running the composer global require laravel/installer

- Git installation,
- modifying the php.ini file,
- or environment configuration.

Problem:

- Composer requires Git to download dependencies (like laravel/installer) from repositories. If Git is not installed or not properly set up, you'll see errors like:
Copy code
- Failed to clone ... repository not found
- Need XAMPP Latest Version
- composer clear –cache on terminal
- Then install composer

Solution:

- **Verify Git Installation:**

- Run `git --version` in the terminal to check if Git is installed.
- Install Git: On Windows: Download Git from git-scm.com and follow the installation instructions.

- **Modifying php.ini File:**

- The `php.ini` file often needs to be updated, which involves removing the semicolon (;) in front of specific lines to enable extensions.

`extension=zip`

Remove ; from above line

Running a Laravel app

cd first-app

php artisan serve

```
PS D:\Laravel Projects> cd first
```

```
PS D:\Laravel Projects\first> php artisan serve  
forking is not supported on this platform
```

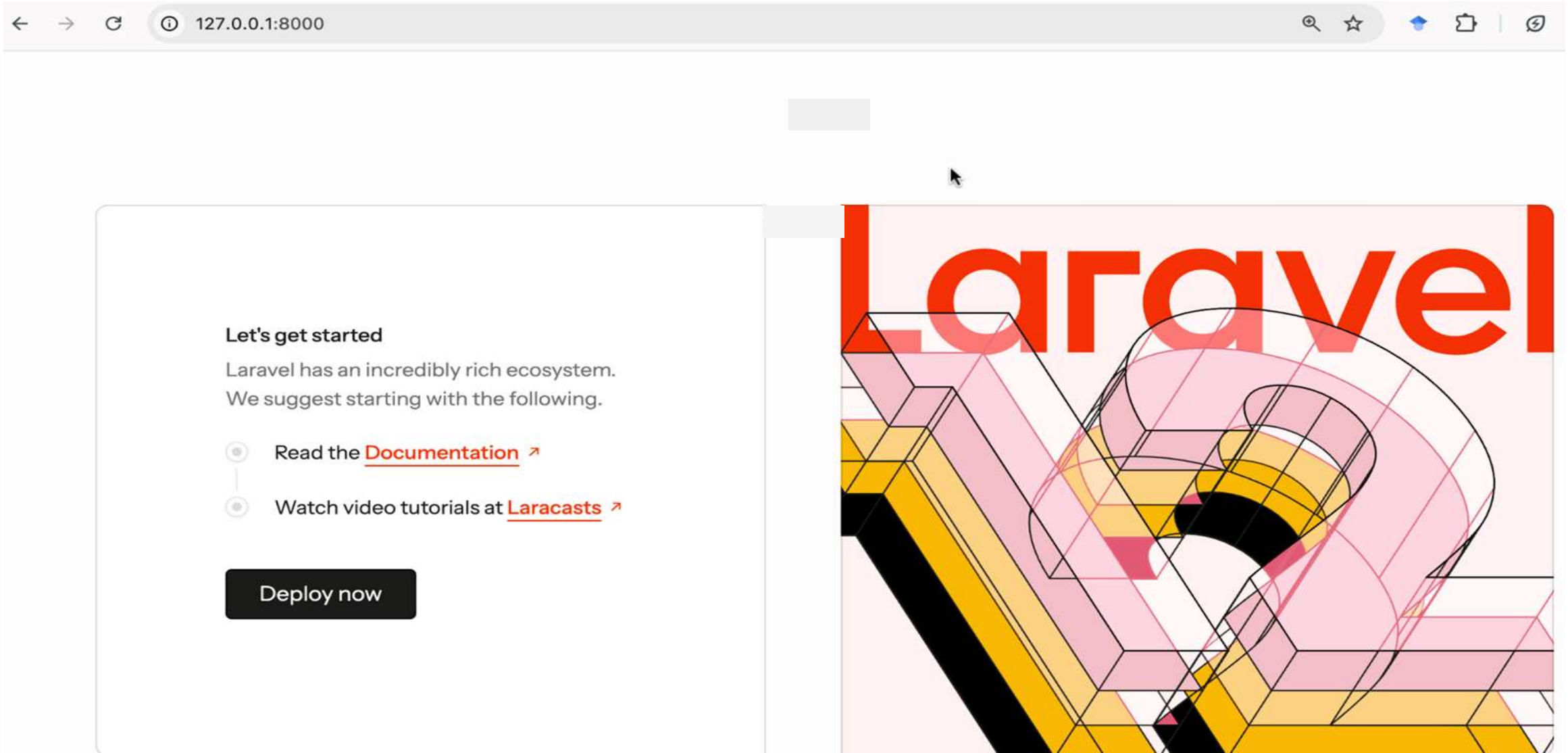
```
INFO Server running on [http://127.0.0.1:8000].
```

```
Press Ctrl+C to stop the server
```

```
PS D:\Laravel Projects\first> php artisan make view::data  
Laravel Framework 11.37.0
```

- The artisan serve command in Laravel is a way to start a built-in PHP development server for your Laravel project.
- This command is commonly used during local development to quickly serve your application without needing to set up a full web server like Apache.
- Starts a local development server at `http://127.0.0.1:8000` by default.
- It allows developers to test their Laravel application locally in a browser.
- Default Port Number: 8000

Laravel Home Page



Artisan

- Command line interface used in Laravel is called Artisan.
 - It includes a set of commands which assists in building a web application.
 - Artisan is Laravel's **command-line interface (CLI)** that provides powerful commands for automating tasks like database migrations, model generation, route listing, and caching.
 - It helps developers save time by reducing manual work.
-
- `php artisan list`
 - a. Displaying All Artisan Commands

A. Application Management

Command	Description
<code>php artisan serve</code>	Starts a development server at <code>http://127.0.0.1:8000/</code>
<code>php artisan down</code>	Puts the application into maintenance mode
<code>php artisan up</code>	Brings the application back online
<code>php artisan about</code>	Displays Laravel version & system info

B. Route Management

Command	Description
<code>php artisan route:list</code>	Displays all registered routes
<code>php artisan route:cache</code>	Caches routes for faster performance
<code>php artisan route:clear</code>	Clears the cached routes

C. Model & Database Commands

Command	Description
<code>php artisan make:model Product</code>	Creates a new model (<code>app/Models/Product.php</code>)
<code>php artisan make:model Product -m</code>	Creates a model with a migration file
<code>php artisan make:migration create_products_table</code>	Generates a migration file
<code>php artisan migrate</code>	Runs all pending migrations
<code>php artisan migrate:rollback</code>	Rolls back the last migration
<code>php artisan db:seed</code>	Runs database seeders
<code>php artisan make:seeder ProductSeeder</code>	Creates a new database seeder

D. Authentication & Authorization

Command	Description
<code>php artisan make:auth</code>	Generates authentication scaffolding (for Laravel 5.x, 6.x)
<code>php artisan make:policy PostPolicy</code>	Creates a new policy for authorization

E. Caching & Performance Optimization

Command	Description
<code>php artisan cache:clear</code>	Clears the application cache
<code>php artisan config:cache</code>	Caches configuration files
<code>php artisan view:cache</code>	Caches Blade templates
<code>php artisan view:clear</code>	Clears the cached views

F. Queues & Jobs

Command	Description
<code>php artisan queue:work</code>	Starts processing queued jobs
<code>php artisan queue:listen</code>	Listens for new jobs in the queue
<code>php artisan queue:failed</code>	Displays failed jobs
<code>php artisan queue:retry all</code>	Retries all failed jobs

G. Event & Listener Commands

Command	Description
<code>php artisan event:list</code>	Lists all registered events
<code>php artisan make:event OrderShipped</code>	Creates a new event class
<code>php artisan make:listener SendOrderNotification</code>	Creates an event listener

What does MVC stand for in software development?

- a) **Model-View-Controller**
- b) Most Valuable Code
- c) Managed View Components
- d) Modular-Visual-Creation

Always Remember.....

MVC is indeed a **broader design pattern** that defines the high-level architecture of an application, emphasizing the separation of concerns. However, when implementing the MVC pattern, **the actual code is organized into separate layers to achieve modularity, maintainability, and scalability.**

What is the primary purpose of the Model in the MVC architecture?

- a) Handles user interactions and presentation logic
- b) **Represents the data and business logic of the application**
- c) Renders the user interface and displays data
- d) Manages the communication between the Model and View

What is Laravel?

- a) A programming language
- b) A JavaScript library
- c) **A PHP web application framework**
- d) A database management system

Do you know?

Originally, PHP stood for "**Personal Home Page**," as it was created to manage Rasmus Lerdorf's personal website in 1994.

Which command is used to create a new Laravel project using Composer?

- a) **composer create-project Laravel app-name**
- b) composer new laravel/laravel
- c) **laravel new app-name**
- d) composer new-project laravel/laravel

What is Eloquent in Laravel?

- a) A templating engine
- b) A database management system
- c) A PHP extension for performance optimization
- d) **An ORM (Object-Relational Mapping) for database interactions**

```
@Entity
@Table(name="addresses")
public class Address {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="id")
    private Long id;
```

```
    @Column(name="street")
    private String street;
```

```
    @Column(name="city")
    private String city;
```

```
    @Column(name="state")
    private String state;
```

```
    @Column(name="country")
    private String country;
```

```
    @Column(name="zip_code")
    private String zipCode;
```

```
Address address = new Address();
address.setCity("Pune");
address.setStreet("Kothrud");
address.setState("Maharashtra");
address.setCountry("India");
address.setZipCode("411047");
```



Result Grid

Filter Rows: Search

id	city	country	state	street	zip_code
2	Pune	India	Maharashtra	Kothrud	411047
3	Pune	India	Maharashtra	Kothrud	411047

Directory Structure

The default Laravel application structure is intended to provide a great starting point for both large and small applications.

But you are free to organize your application however you like.

▼ first

- > app
- > bootstrap
- > config
- > database
- > public
- > resources
- > routes
- > storage
- > tests
- > vendor
- ⚙ .editorconfig
- ⚙ .env
- \$.env.example
- 💎 .gitattributes
- 💎 .gitignore
- ≡ artisan
- { } composer.json
- { } composer.lock
- { } package.json
- 📡 phpunit.xml
- ℹ README.md
- JS vite.config.js

App -> Http

This directory houses the controllers, middleware, and form requests. **Controllers handle HTTP requests and responses, middleware provides a way to filter incoming HTTP requests, and form requests handle form validation logic.**

```
✓ app
  > Http
  > Models
  > Providers
```

App -> Models

In this directory, you define the **Eloquent ORM models**. Eloquent is Laravel's built-in ORM that allows you to interact with the database using an expressive syntax.

Providers

- Stores service provider classes.
- Service providers are responsible for bootstrapping your application.
- By default, Laravel includes several providers, such as AppServiceProvider, AuthServiceProvider, and EventServiceProvider.

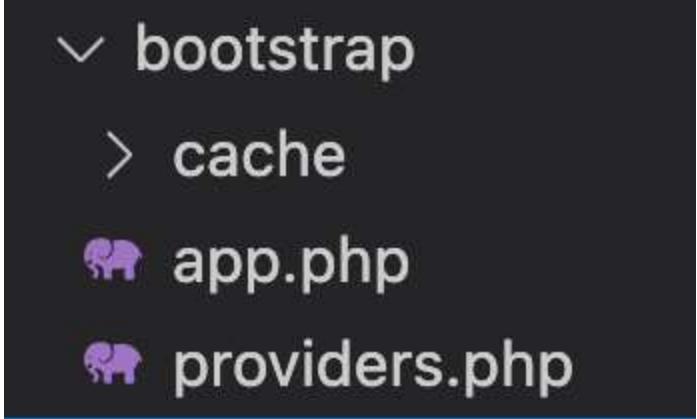
Do you know ?

Laravel drew inspiration from several other frameworks, including **Ruby on Rails, ASP.NET MVC, and Sinatra**. It combines the best features from these frameworks and adds its own unique elements.

The Bootstrap Directory

The bootstrap directory contains files responsible for bootstrapping the Laravel application.

- app.php: This file loads the basic application configuration and **creates an instance of the Laravel application**.
- cache: The cache directory contains compiled service container and configuration files. These files improve the **application's performance by reducing the bootstrapping time**.



```
✓ bootstrap
  > cache
  🐘 app.php
  🐘 providers.php
```

In a standard Laravel project, which directory contains the main application logic and business-specific code?

a) resources

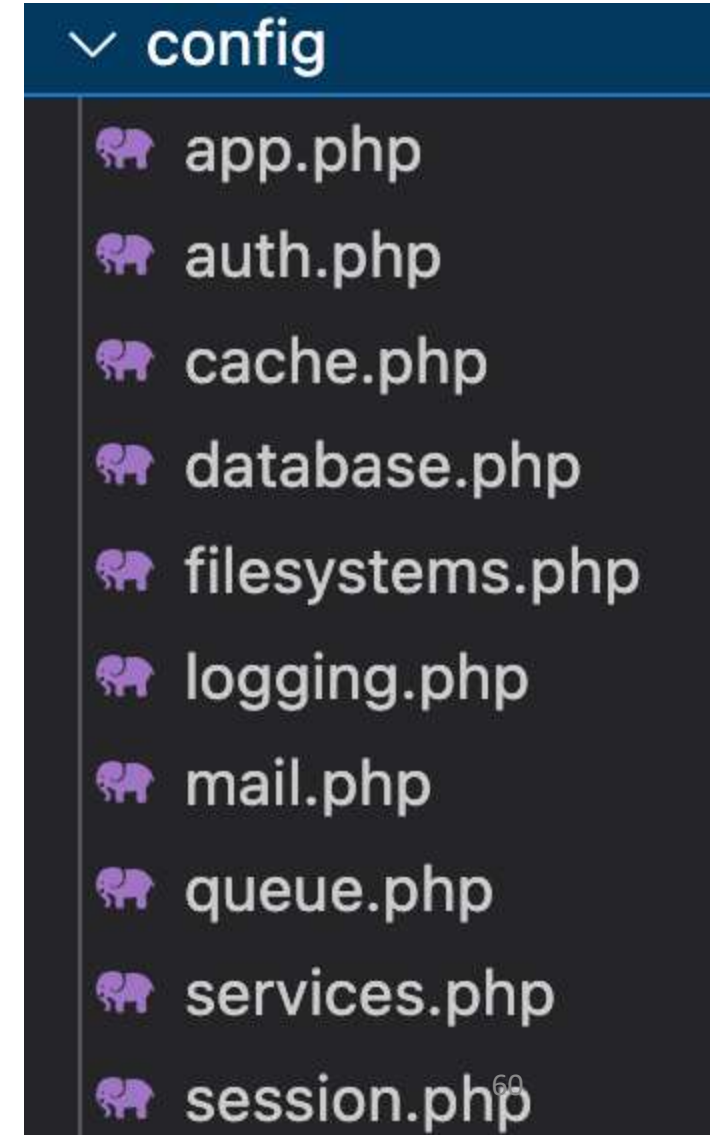
b) public

c) app

d) database

Config Directory

The config directory houses various **configuration files for different components of the Laravel application**. You can customize settings like database connections, application settings, cache configurations, and more from these files.



Database Directory

The database directory contains **database-related files**, including migrations, seeds, and factories.

```
✓ database
  > factories
  > migrations
  > seeders
  ◆ .gitignore
  ≡ database.sqlite
```

Database -> Factories

Factory classes are used to define data blueprints that can be **used to create dummy data for testing**.

Database -> Migration

Migrations are **version control for your database**. They allow you to modify the database schema and **keep it in sync with your application code**.

Database -> Seeds

Seed classes **allow you to populate your database with sample data for testing** or initial setup.

In Laravel, where do you define the database schema and migrations for creating and modifying database tables?

a) resources/views

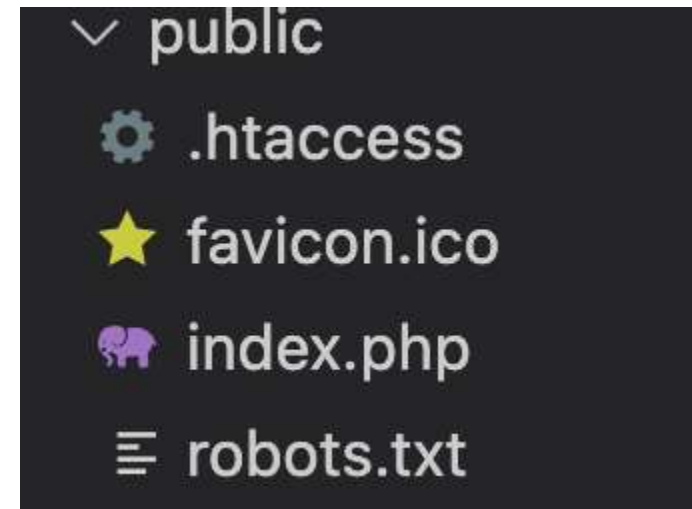
b) app

c) database/migrations

d) config

Public Directory

- The public directory is the web server's document root. It **contains the index.php file, which is the entry point for all HTTP requests.**
- configures autoloading
- It is the directory where all public-facing files reside, such as assets, images, Javascript, CSS and favicon icon and the primary PHP file that handles incoming requests.



Which directory is used for storing publicly accessible assets such as images, stylesheets, and JavaScript files?



- a) resources
- b) **public**
- c) storage
- d) app

Resource Directory

The resources directory contains the views, language files, and assets (CSS, JavaScript, and images) for the application.

- **Views:** The views directory **stores Blade templates** that define the UI of the application.
- **Lang:** The lang directory **holds language files for localization** and internationalization purposes.
- **Assets:** The assets directory **contains raw CSS, JavaScript, and other assets**. These assets are usually compiled and optimized using Laravel Mix.

```
▼ resources
  > css
  > js
  > views
```

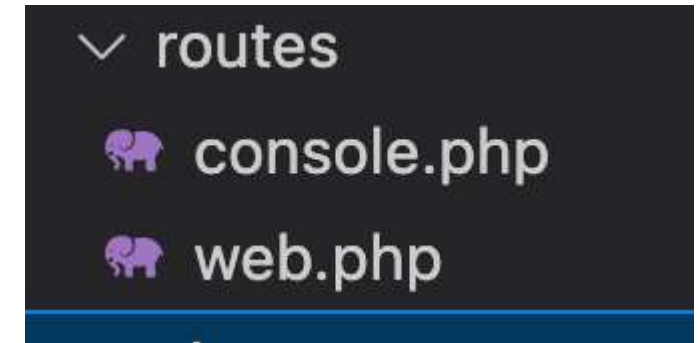
Do you Know?

It takes less than a second (**0.9 seconds** to be precise) for users to form their opinions on their initial visit to a web page.

Routes Directory

The routes directory contains route definition files.

- **web.php**: This file contains routes that are typically associated with web-based user interfaces.
- **api.php**: This file contains routes for API endpoints, usually meant to return JSON responses.



routes/web.php (opened at which link)

- In Laravel, the routes/web.php file is where you define routes for web interfaces of your application.
- These routes typically respond to requests made via a web browser and are grouped within the web middleware group, which provides features such as session handling, CSRF protection, and cookie encryption.
- Basic Route Example:

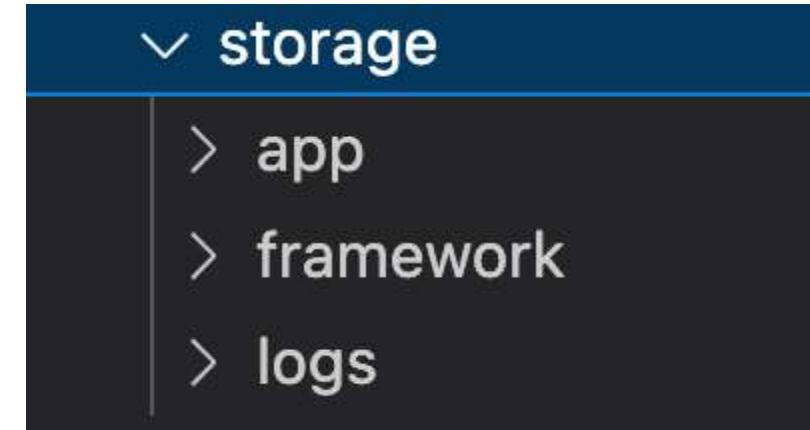
```
Route::get('/', function () {  
    return view('welcome');  
});
```

routes/console.php file (Console commands)

- The routes/console.php file in Laravel is where you define all custom Artisan commands for your application.
- This file allows you to extend and customize the functionality of Laravel's command-line interface (CLI)/terminals.
- Not used directly

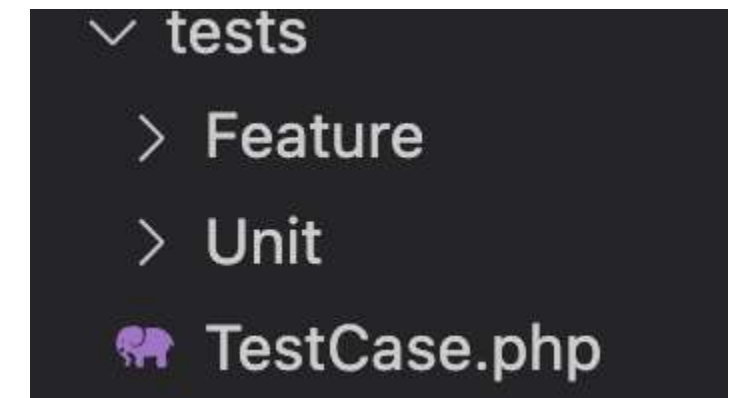
Storage Directory

The storage directory **holds various files generated by the application**, such as logs, cache, sessions, and uploaded files.



Test Directory

The tests directory **contains test cases** for the application.



Vendor Directory

The vendor directory
houses all the third-party dependencies installed via Composer, including
Laravel itself.

✓ vendor

- > bin
- > brick
- > carbonphp
- > composer
- > dflydev
- > doctrine
- > dragonmantank
- > egulias
- > fakerphp
- > filp
- > fruitcake
- > graham-campbell

- > theseer
- > tijsverkoyen
- > vlucas
- > voku
-  autoload.php

Vendor folder (mentioned in composer.json and stored in Vendor)

- The vendor folder in a Laravel project is where Composer stores all the **third-party dependencies required for the application.**
- It is automatically generated and managed by Composer and includes **libraries, frameworks, and tools specified in the composer.json file.**

Autoloading:

- The vendor/autoload.php file is the main entry point for Composer's autoloading mechanism.
- Laravel's bootstrap/app.php includes this file to autoload all necessary classes

Organizational Structure:

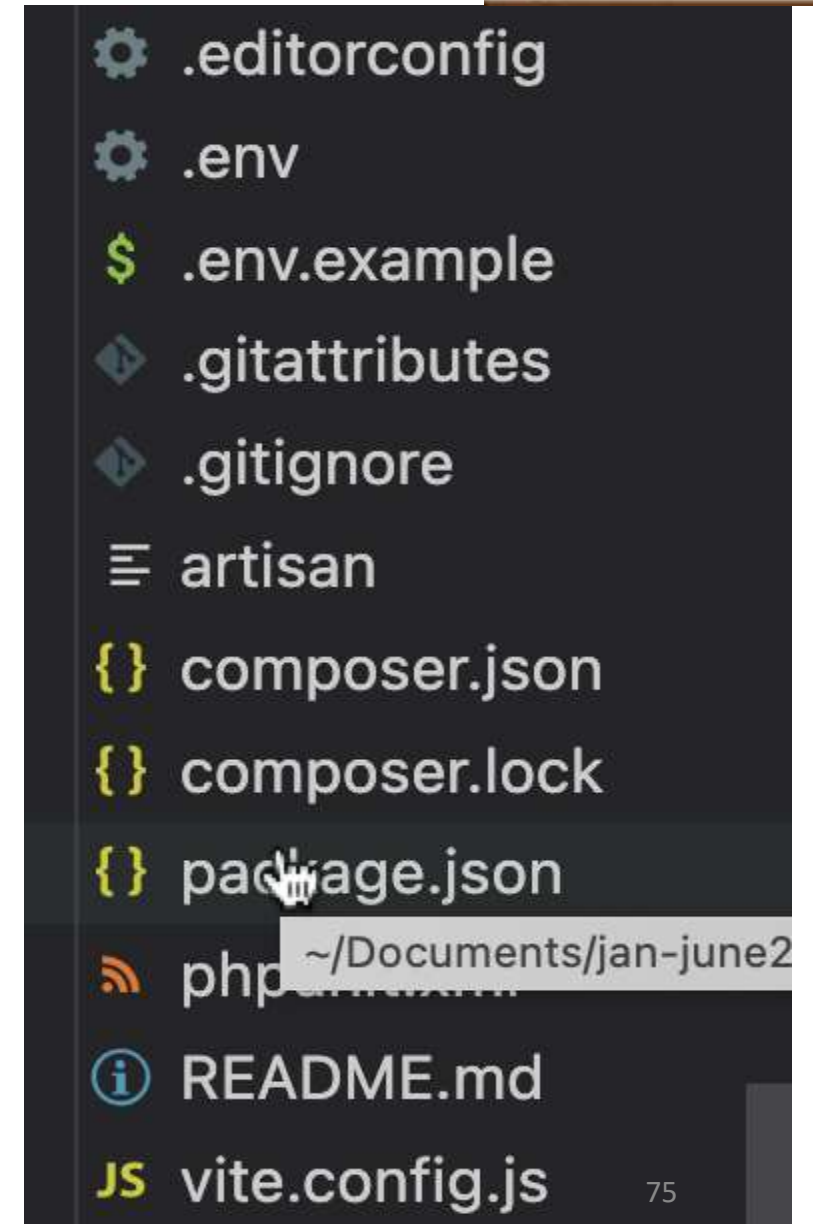
- Dependencies are organized in subdirectories based on their namespaces (e.g., vendor/symfony/, vendor/laravel/).
- It also contains the composer/ directory, which manages autoloading and metadata about the dependencies.

.editorconfig

This file is used to **maintain consistent coding styles** across different editors and IDEs in a project.

It allows you to **change settings without modifying code**

- Stores **database credentials**
- Keeps **API keys secure**
- Helps manage **different environments** (local, production)

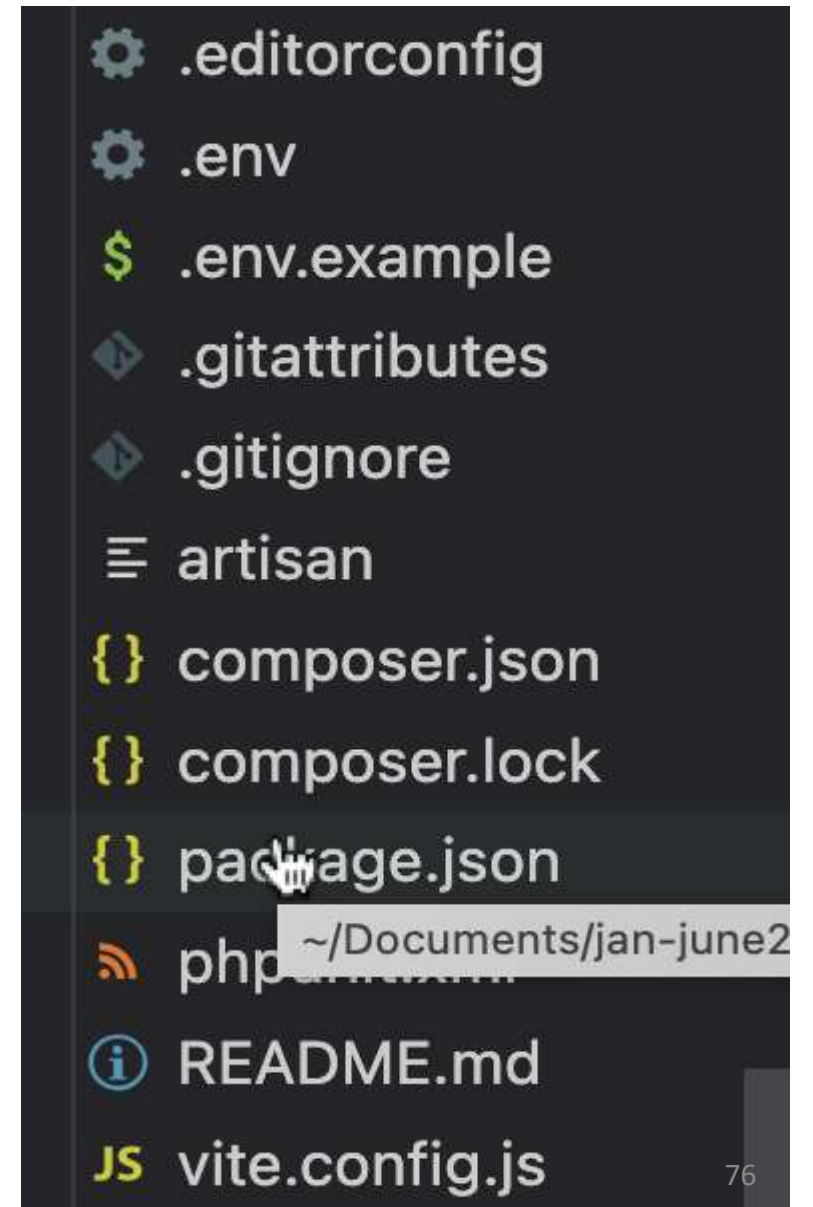


.env

The **.env** (environment file) is used to store **configuration settings and sensitive data** for your application.

It allows you to **change settings without modifying code**

- Stores **database credentials**
- Keeps **API keys secure**
- Helps manage **different environments** (local, production)



.env.example

The **.env.example** file is a **template** of the **.env** file.

It contains **all required configuration keys**, but **without real/sensitive values**

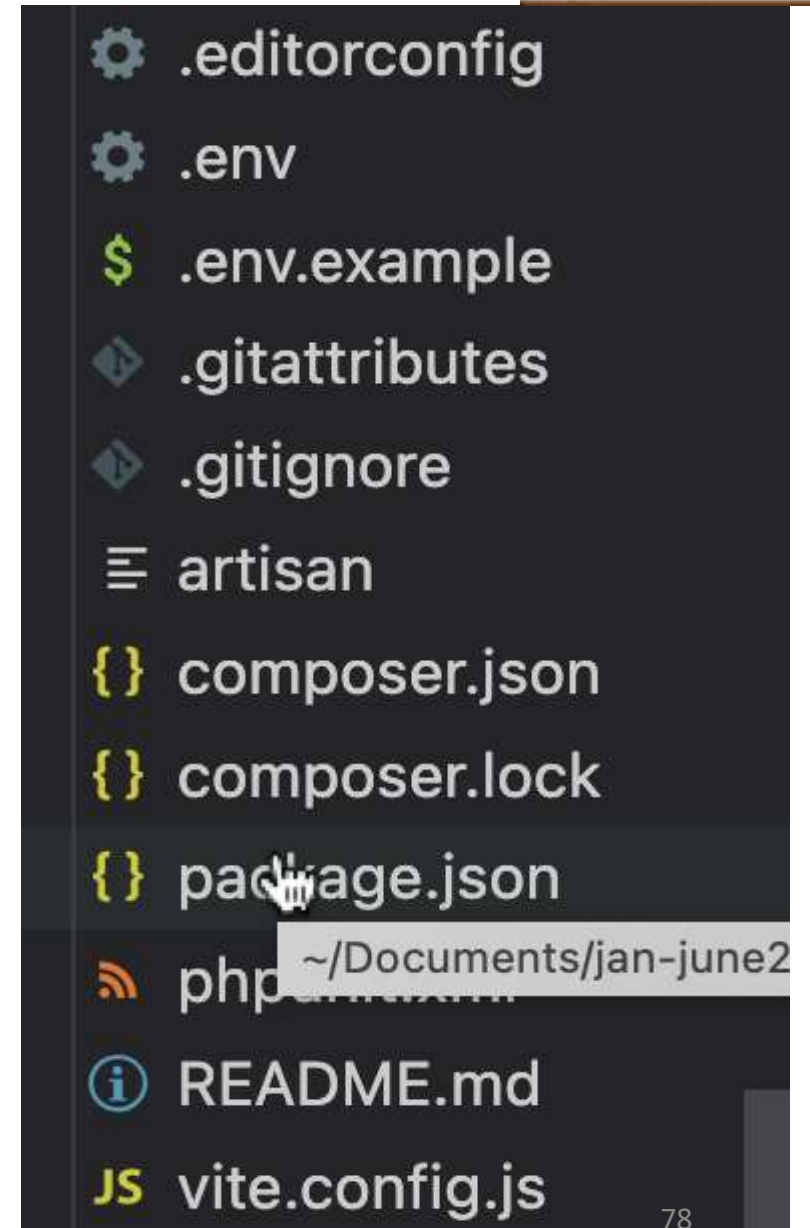
- Helps other developers know **which environment variables are needed**
- Used when setting up a project for the first time
- Keeps **secrets safe** (no passwords or API keys inside)
- It looks almost like **.env**, but values are **dummy/empty**

 .editorconfig .env .env.example .gitattributes .gitignore artisan composer.json composer.lock package.json php ~/Documents/jan-june2 README.md vite.config.js

.gitattributes

The **.gitattributes** file is used by **Git** to **control how files are handled in a repository**.

- Line endings
- File merging behavior
- Exporting files
- Language detection (GitHub)
- Ensures **consistent behavior across systems (Windows/Linux/Mac)**
- Prevents **merge conflicts**
- Helps GitHub correctly **detect languages**
- Controls what files are included in archives



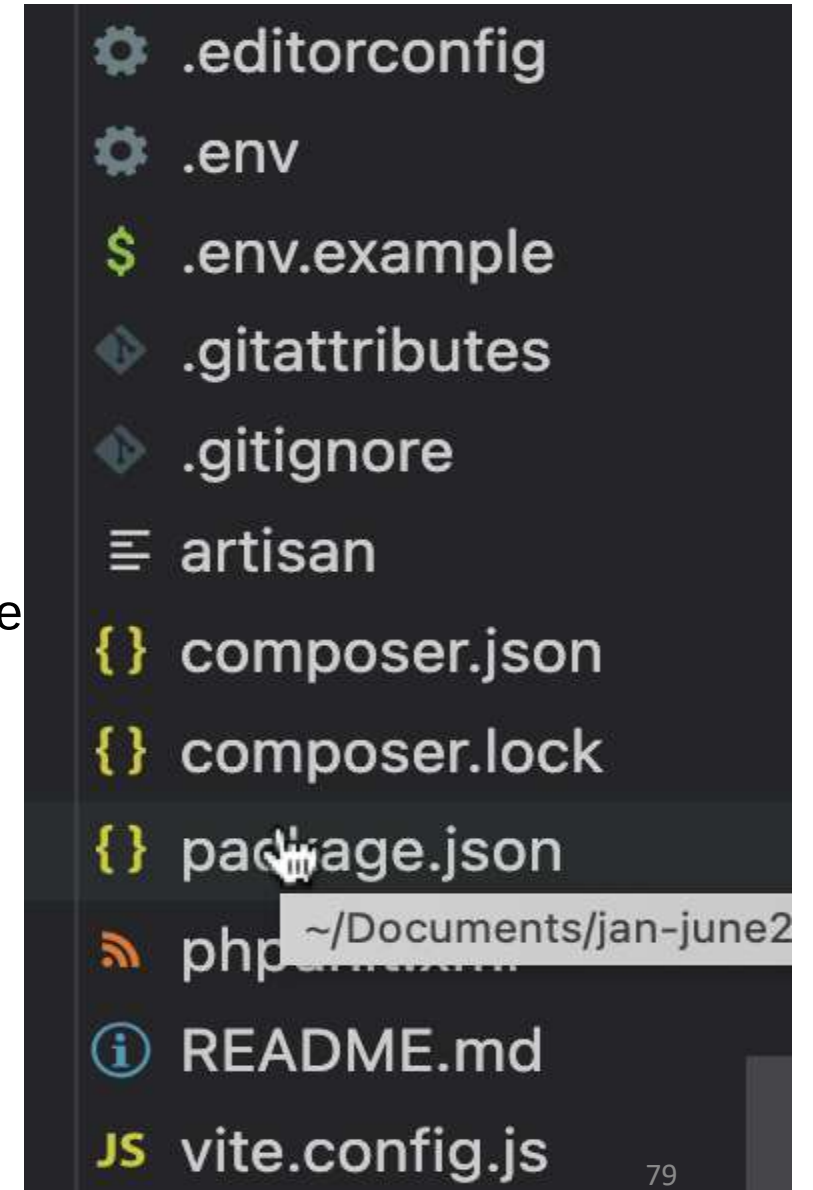
composer.json

The `composer.json` file is the main configuration file for Composer (PHP dependency manager).

- Which **libraries/packages** your project needs
- How to **autoload classes**
- Project **metadata & scripts**

composer.lock

- The `composer.lock` file is generated by Composer and stores the **exact versions of all installed dependencies** in your project.
- While `composer.json` defines *what you want*, `composer.lock` **records what you actually got installed**
- Exact **package versions**
- Dependency tree (sub-dependencies)
- Download sources (URLs)
- Hash for verification



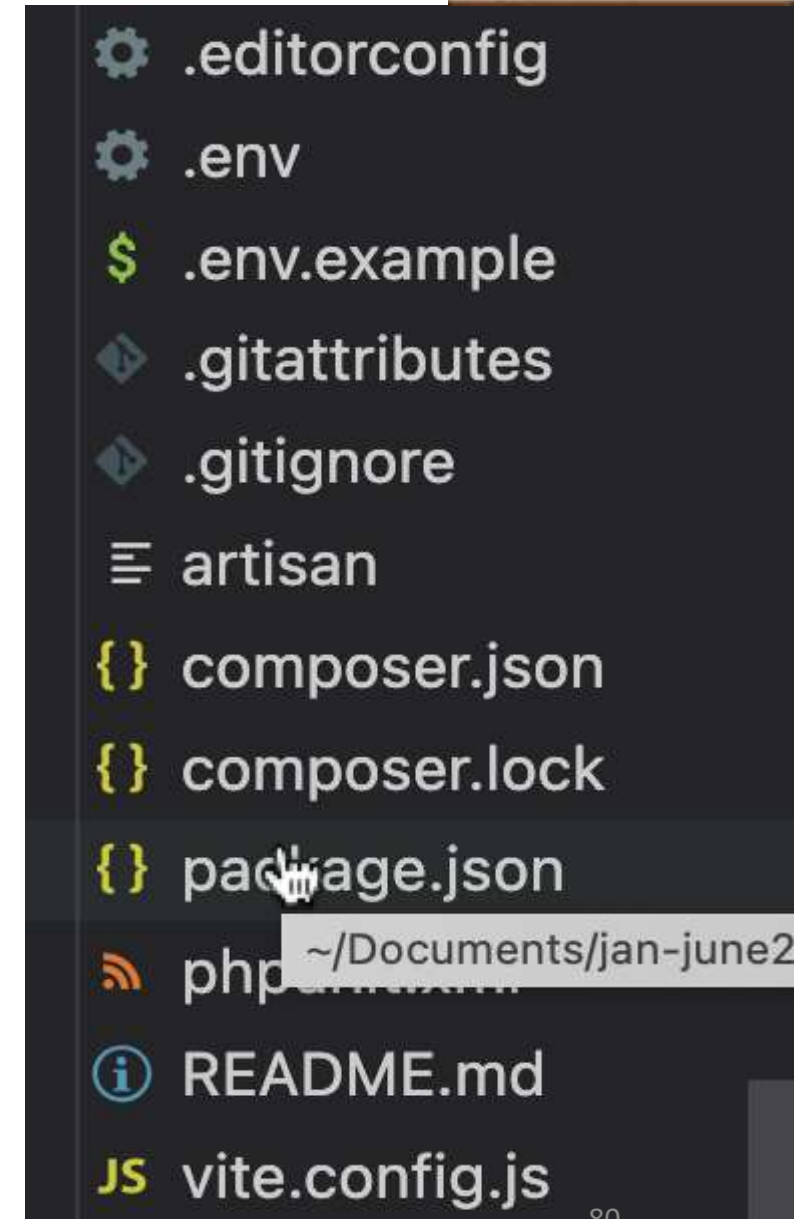
package.json

`package.json` is a file that manages JavaScript dependencies and scripts in a project.

phpunit.xml

The `phpunit.xml` file is a **configuration file** for **PHPUnit**, which is used for **testing in PHP/Laravel**. It defines:

- Test environment settings
- Database config for testing
- Test directories



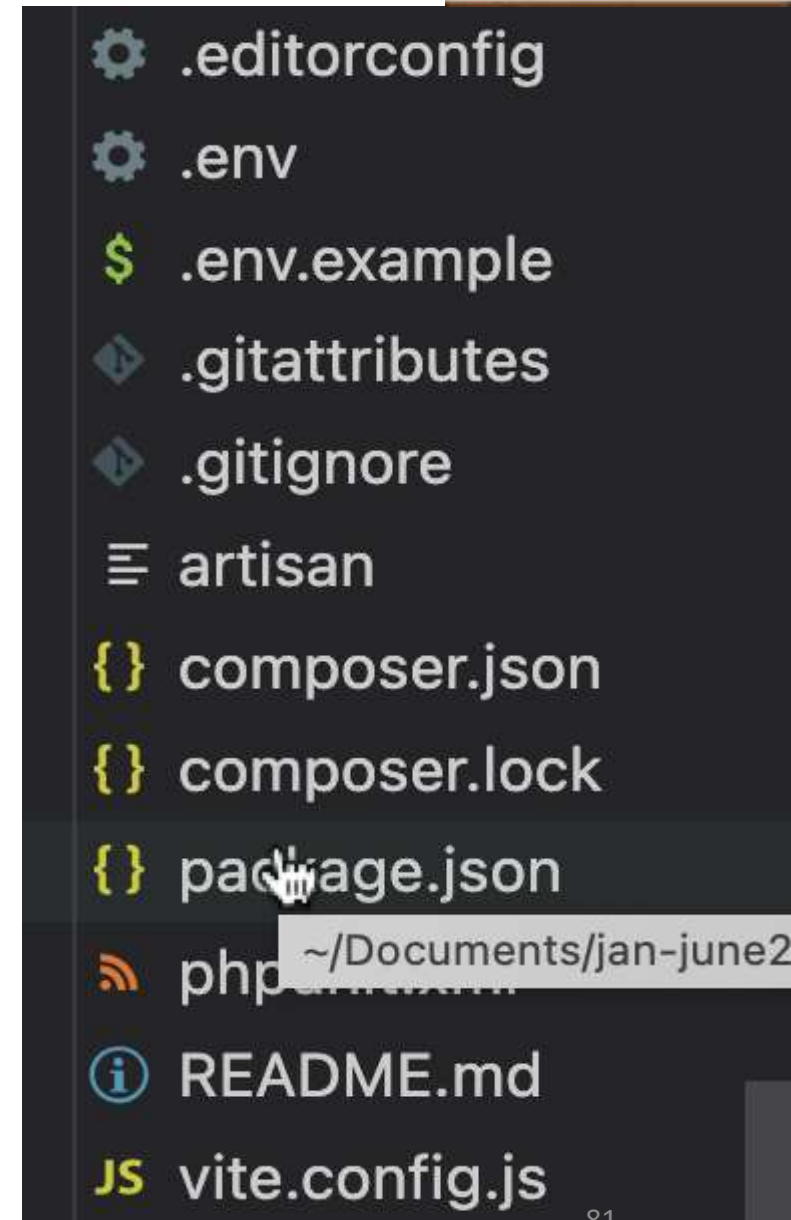
README.md

The **README.md** file is a **documentation file** that explains your project. It is written in **Markdown (.md)** format and is the **first file people see** on GitHub.

vite.config.js

The **vite.config.js** file is a **configuration file** for **Vite**, a modern frontend build tool used in Laravel.

It controls how your **CSS, JavaScript, and assets** are **compiled and served**



In every Laravel project, you will have two files, `composer.json` and `composer.lock`. What is the difference between them?

In composer.json you specify **what packages should be installed, and with what versions.**

```
{  
    "require": {  
        "laravel/breeze": "^1.19",  
    },  
}
```

Now, **which exact version is installed at any moment?** Here comes the composer.lock file.

```
{  
    // ...  
    "name": "laravel/breeze",  
    "version": "v1.19.1",  
    "source": {  
        "type": "git",  
        "url": "https://github.com/laravel/breeze.git",  
        "reference": "4bbb1ea3476901c4f5fc706f8d80e4eac31c3afb"  
    },  
    "dist": {  
        "type": "zip",  
        "url": "https://api.github.com/repos/laravel/breeze/zipball/4bbb1ea3476901c4f5fc706f8d80e4eac31c3afb"  
    }  
}
```

Which directory contains the views (templates) used for rendering HTML pages in Laravel?

a) resources

b) public

c) app

d) routes

Which directory is used for storing files generated by the application, such as logs, cache, and session data?

a) resources

b) public

c) storage

d) config